

GPU Computing

Exercise 7

Luis Wenzel
Miro Joensuu
Daniel Bayer
Noah Schneider

1 Presenting

Willing to present:

7.1 Yes

7.2 Yes

7.3 Yes

2 Naive GPU Implementation

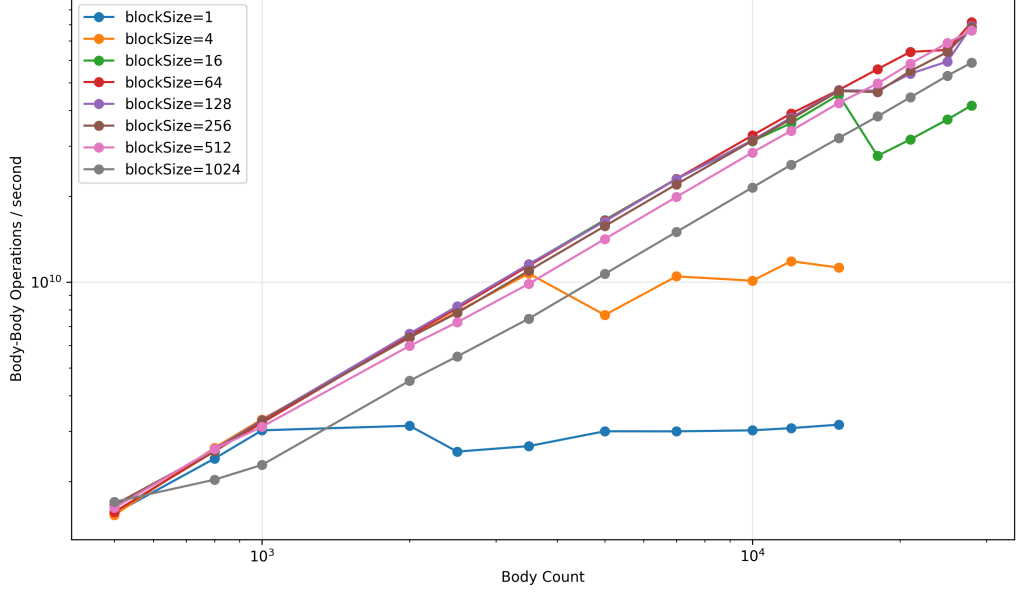


Figure 1: Naive GPU Performance (500 iterations)

Plot 1 shows the performance of the naive GPU implementation over 500 iterations. The measurements were taken as median values to account for outliers.

Performance increases with larger block sizes, as more warps can be scheduled concurrently. For block sizes of 256 and higher, performance is quite similar, as the GPU cores are well utilized. For smaller block sizes, performance drops significantly for larger matrices, since the CUDA cores are underutilized.

This means that the overall performance (interactions per second) also increases with a larger problem size, up to the point where a limit for the current block size is reached. From there, performance stays roughly constant.

We can also see that the larger block sizes (512 and 1024) perform slightly worse than the 64-256 range, which might be due to a reduced flexibility of scheduling.

3 Optimized GPU Implementation

With regards to using SOA vs AOS: It makes sense to use SOA for this use case, because when we fill the shared memory, we only need to read the position and mass data of each body. A SOA layout allows us to read only those properties, without having to deal with the velocity data, which would otherwise be interleaved in an AOS layout. There might be a penalty for the update of the position, since the position data is stored separately from the velocity data, but this will be negligible compared to the performance gain on the force calculation.

We thus implemented the optimized version using a packed SOA data structure. We kept the Block Size constant at 128 for this and the following exercises, only varying the particle number. We used 1000 iterations for each simulation.

In this version (picture2) we also implemented tiling. This means we process

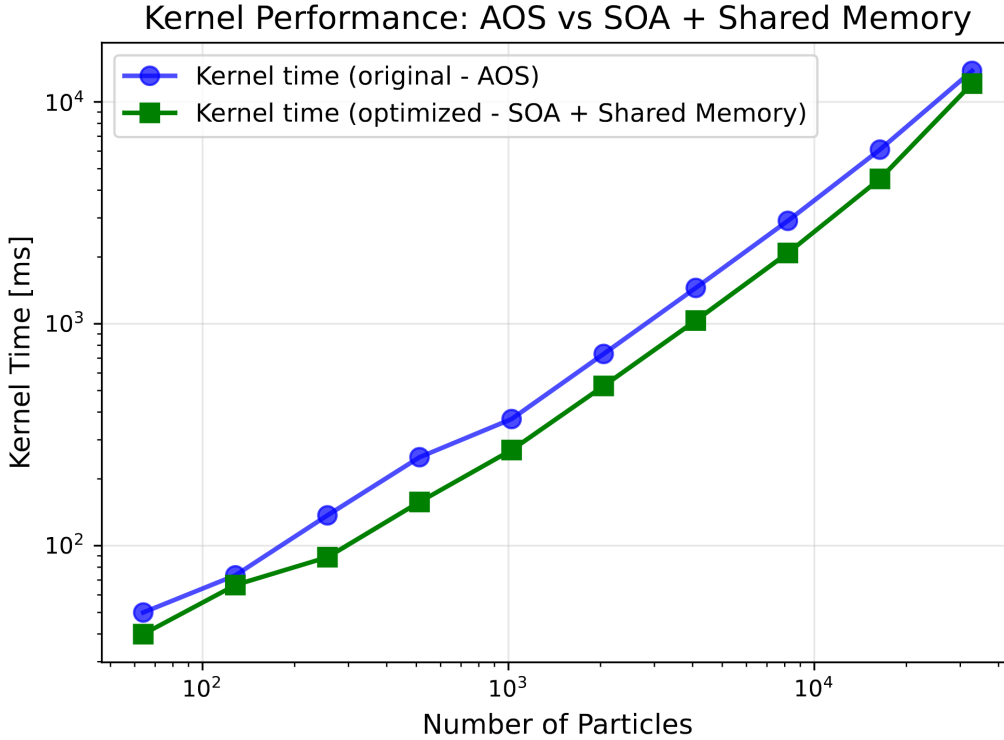


Figure 2: Kernel Performance: AOS vs SOA + Shared Memory

small blocks of particles at a time, each one fitting into shared memory. We

calculate the forces from this “source” block on all our “target” particles (one assigned to each thread) and then swap out for the next “source” block. The “source” blocks are always loaded into shared memory and shared between all threads in a block. Through this we can utilize the faster shared memory for our computations before moving on to the next block. We address global memory less, improving our speedup.

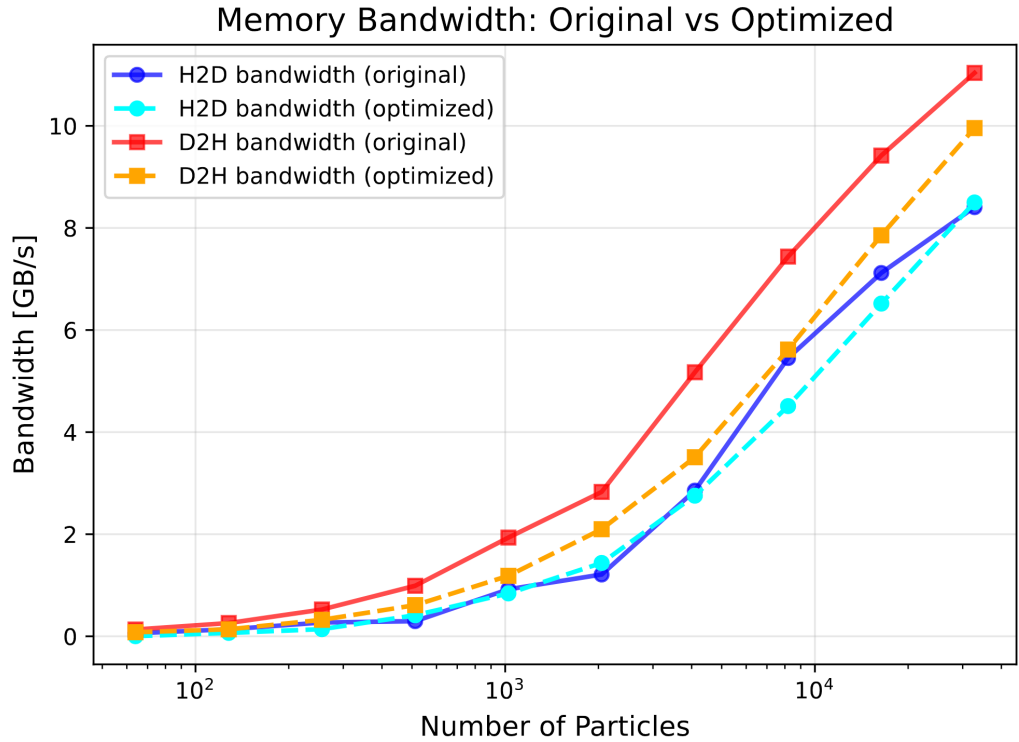


Figure 3: Memory Bandwidth: Original vs Optimized

Using both these optimizations we see a improvement in the kernel time, which is the most significant component of the total execution time, dominating both the d2h and h2d times (these are largely unaffected by the optimization, as no changes were made (picture 3)). We reach a maximum speedup of 1.59x against the benchmark and an average improvement of 1.35x over the particle numbers we tested.

Using loop unrolling within our kernels can further increases our performance, but only slightly. This is because the loops can already be pre-

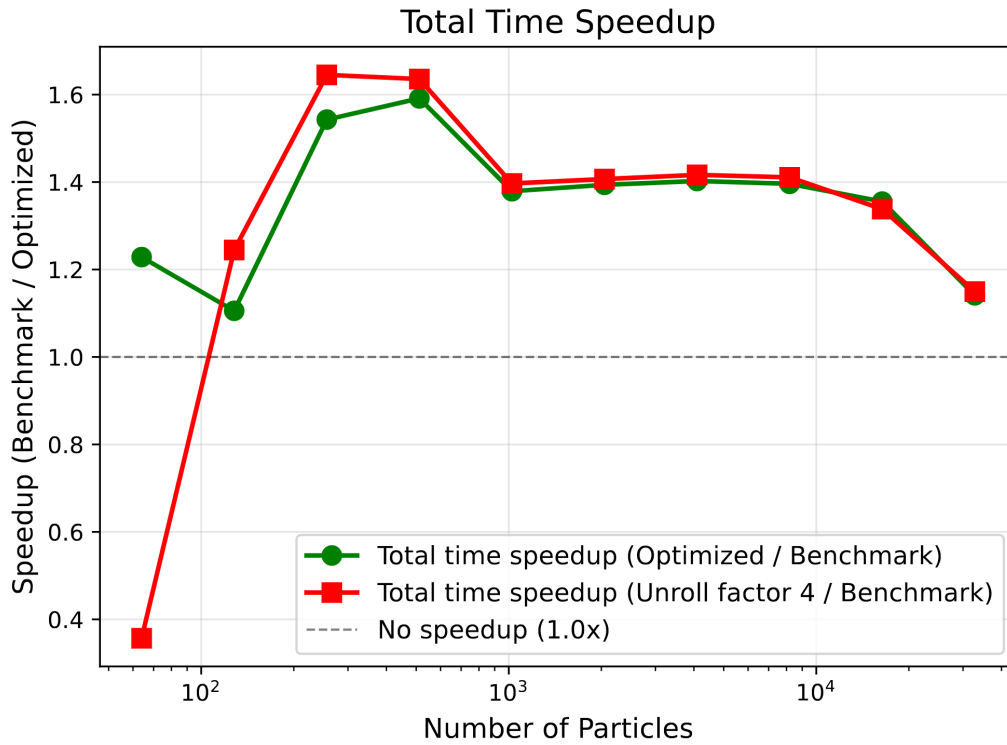


Figure 4: Speedup in Optimized and Uroll factor 4

processed at compile time and thus can reduce overhead during runtime. It can however, also lead to a decrease in performance due to higher register pressure and increased code size. Using loop unrolling we increase our maximum speedup to 1.64x (picture 4) over the benchmark, but decrease our average to 1.30x. How effective loop unrolling is, seems therefore to depend on the particle number.

4 n-Body GPU computations and streaming

In this exercise we limited our GPUs memory to 4MB and used a particle number of 512k which comes out to about 16MB for an assumed 32 bytes per particle, which leaves some room for data overhead. The code was implemented using the host memory as a swap space. We firstly load a batch of target particles (positions and velocities), i.e. those onto which forces will be accumulated. We then load batches of source particles (only positions) and accumulate forces from those onto our target particles, until the target particles have a full velocity update and we can update their positions. We then load the next batch of target particles and repeat.

This algorithm is actually quite similar to the tiled approach we already use in our optimized kernel calculation. The difference being that the kernel assigns all target particles it is served a thread and only further subdivides the source particles into smaller tiles due to exploit shared memory. In our course grained algorithm both target and source particles have to be batched due to the global memory constraints we imposed.

We can additionally overlap some parts of communications and computations using streaming. For example while the position update is being conducted on the target particles we can already write back their velocities to the host. We can also load the target and source particles in parallel. The amount of parallelism we can conduct, at least in our implementation is limited, since both data loads and computations mostly depend on each other and thus cannot be parallelised.

The small effect of streaming can be seen in our results (picture). It only leads to a very small speedup, especially at smaller segment sizes, over our non-streamed version (using only stream #0).

In our implementation we are using three streams, although by using even more fine grained data accesses (i.e. loading and computing two source particle batches simultaneously) could potentially increase the gain from streaming. This would however also increase computational overhead, as we would have to sum up the velocity updates from both batches before we update the velocities of the target particles. This would have to be tested separately in practice.

The surprising finding is that our “limited” version is much faster than our optimized benchmark (at a particle size of 512k), with or without streaming. This is likely due the large particle number being much more efficiently

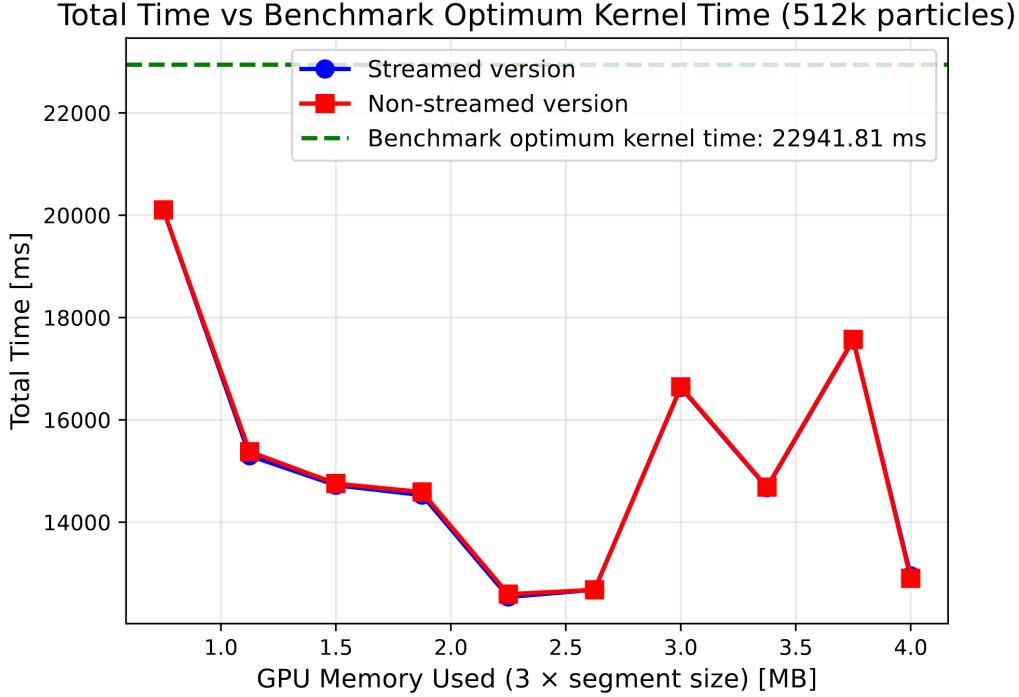


Figure 5: Total Time vs Benchmark Optimum Kernel Time by 512k particles

processed when it is batched. Without batching one huge kernel has to be launched at once to process all the particles and here we can reduce its overhead by subdividing it into many smaller kernel launches. This improvement seems to dominate even though we move much more data between device and host, which makes sense as the kernel time dominated the h2d and d2h time already previously. At this particle number the improvement from our optimized version over the benchmark drops to 1.07x (only looking at kernel time)(picture 6). The maximum speedup (with a concurrent memory usage of 2MB on the GPU) is 1.95x (using streams) and 1.94x (without streams). The average is 1.65x (using streams) and 1.64x (without streams).

For different segment sizes we get a different total memory usage on the GPU and also increase the size of kernels launched. We see an increase up until around 2MB memory used, likely because the lower data movement and fewer kernels launched (less launch overhead) is faster. After that we see a decrease around 3MB, likely because the particle number doesn't match well with the segment size and we launch additional kernels for only a few leftover particles. We then see another increase around the maximum 4MB almost back to what we saw at 2MB. As we further increase the segment size we

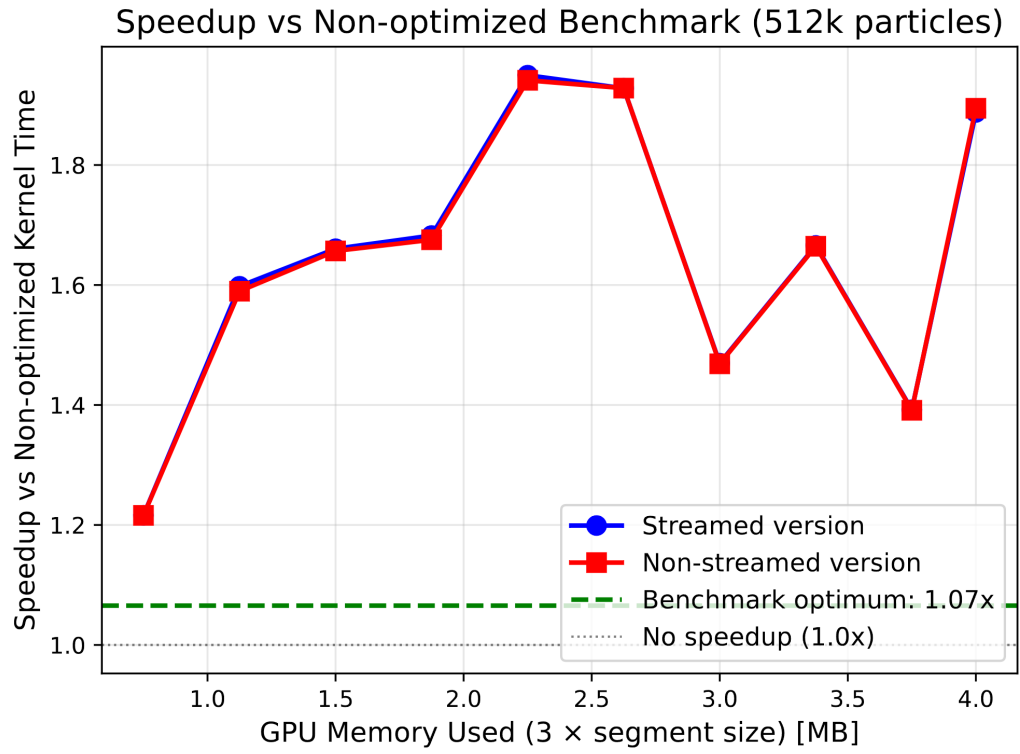


Figure 6: Speedup vs Non-optimized Benchmark 512k particles

would eventually expect a drop down to the speed of our “optimized” version as the overhead from launching very large kernels would start to dominate.